

Revision Notes

A clean interview-focused handbook for revising patterns, complexities, and solutions without visual clutter.

SOLVED

31

PATTERNS

8

LANGUAGES

1

Difficulty split

Easy

11

Medium

17

Hard

3

HOW TO REVISE

1. Read the title and question. 2. Cover the code and recall the pattern. 3. Check time and space complexity. 4. Only then inspect the solution.

The goal is recall, not rereading. Try to solve the idea in your head before the code.

Revision Index

Use this page to jump to a problem by number, pattern, or difficulty.

0001	1. Two Sum	Hashing / Set	Easy
0002	2. Add Two Numbers	Linked List	Medium
0003	3. Longest Substring Without Repeating Characters	Hashing / Set	Medium
0004	4. Median of Two Sorted Arrays	Sorting	Hard
0007	7. Reverse Integer	Math / Simulation	Medium
0009	9. Palindrome Number	Math / Simulation	Easy
0011	11. Container With Most Water	Two Pointers	Medium
0014	14. Longest Common Prefix	Arrays / Implementation	Easy
0019	19. Remove Nth Node From End of List	Linked List	Medium
0021	21. Merge Two Sorted Lists	Linked List	Easy
0024	24. Swap Nodes in Pairs	Linked List	Medium
0033	33. Search in Rotated Sorted Array	Arrays / Implementation	Medium
0034	34. Find First and Last Position of Element in Sorted Array	Binary Search	Medium
0035	35. Search Insert Position	Binary Search	Easy
0049	49. Group Anagrams	Hashing / Set	Medium
0053	53. Maximum Subarray	Greedy / Kadane	Medium
0076	76. Minimum Window Substring	Hashing / Set	Hard
0081	81. Search in Rotated Sorted Array II	Binary Search	Medium
0094	94. Binary Tree Inorder Traversal	Binary Search	Easy
0098	98. Validate Binary Search Tree	Binary Search	Medium
0100	100. Same Tree	Binary Search	Easy
0101	101. Symmetric Tree	Binary Search	Easy
0121	121. Best Time to Buy and Sell Stock	Greedy / Kadane	Easy
0122	122. Best Time to Buy and Sell Stock II	Greedy / Kadane	Medium
0128	128. Longest Consecutive Sequence	Hashing / Set	Medium
0189	189. Rotate Array	Math / Simulation	Medium
0209	209. Minimum Size Subarray Sum	Arrays / Implementation	Medium
0215	215. Kth Largest Element in an Array	Arrays / Implementation	Medium
0239	239. Sliding Window Maximum	Two Pointers	Hard
0455	455. Assign Cookies	Sorting	Easy
3718	4080. Smallest Missing Multiple of K	Hashing / Set	Easy

PROBLEM 0001

1. Two Sum

Easy

Hashing / Set

C++

QUESTION

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

EXAMPLE

Input: `nums = [2,7,11,15]`, `target = 9` Output: `[0,1]` Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

PATTERN

Hashing / Set

TIME

$O(n)$ average

SPACE

$O(n)$

SOLUTION

C++ • 0001

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> mp;
        for (int i = 0; i < nums.size(); i++) {
            int x = target - nums[i];
            if (mp.find(x) != mp.end()) {
                return {mp[x], i};
            }
            mp[nums[i]] = i;
        }
        return {};
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

1 / 31

PROBLEM 0002

2. Add Two Numbers

Medium

Linked List

C++

QUESTION

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

EXAMPLE

Input: l1 = [2,4,3], l2 = [5,6,4] Output: [7,0,8] Explanation: 342 + 465 = 807.

PATTERN
Linked List

TIME
O(n)

SPACE
O(1)

SOLUTION

C++ • 0002

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode* dummy = new ListNode(0);
        ListNode* temp = dummy;
        int carry = 0;
        while (l1 != NULL || l2 != NULL || carry != 0) {
            int sum = carry;
            if (l1 != NULL) {
                sum += l1->val;
                l1 = l1->next;
            }
            if (l2 != NULL) {
                sum += l2->val;
                l2 = l2->next;
            }
            carry = sum / 10;
            temp->next = new ListNode(sum % 10);
            temp = temp->next;
        }
        return dummy->next;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

2 / 31

PROBLEM 0003

3. Longest Substring Without Repeating Characters

Medium

Hashing / Set

C++

QUESTION

Given a string *s*, find the length of the longest substring without duplicate characters.

EXAMPLE

Input: *s* = "abcabcbb" Output: 3 Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

PATTERN

Hashing / Set

TIME

$O(n)$ average

SPACE

$O(n)$

SOLUTION

C++ • 0003

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_set<char> st;
        int left = 0;
        int maxi = 0;
        for (int right = 0; right < s.size(); right++) {
            while(st.find(s[right]) != st.end()) {
                st.erase(s[left]);
                left++;
            }
            st.insert(s[right]);
            maxi = max(maxi, right - left + 1);
        }
        return maxi;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

3 / 31

PROBLEM 0004

4. Median of Two Sorted Arrays

Hard

Sorting

C++

QUESTION

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

EXAMPLE

Input: `nums1 = [1,3]`, `nums2 = [2]` Output: `2.00000` Explanation: merged array = `[1,2,3]` and median is 2.

PATTERN
Sorting

TIME
 $O(n \log n)$

SPACE
 $O(n) / O(1)$

SOLUTION

C++ • 0004

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        nums1.insert(nums1.end(), nums2.begin(), nums2.end());
        sort(nums1.begin(), nums1.end());
        if(nums1.size()%2==0){
            return (nums1[(nums1.size()/2)-1]+nums1[nums1.size()/2])/2.0;
        }
        else{
            return nums1[nums1.size()/2.0];
        }
        return{};
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

4 / 31

PROBLEM 0007

7. Reverse Integer

Medium

Math / Simulation

C++

QUESTION

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

EXAMPLE

Input: $x = 123$ Output: 321

PATTERN
Math / Simulation

TIME
 $O(n)$

SPACE
 $O(1)$

SOLUTION

C++ • 0007

```
class Solution {
public:
    int reverse(int x) {
        int rev = 0;

        while(x!=0){
            int d = x%10;
            if(rev > INT_MAX/10 || rev < INT_MIN/10)
                return 0;
            rev = rev*10 + d;
            x=x/10;
        }
        return rev;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

5 / 31

9. Palindrome Number

Easy

Math / Simulation

C++

QUESTION

Given an integer x , return true if x is a palindrome, and false otherwise.

EXAMPLE

Input: $x = 121$ Output: true Explanation: 121 reads as 121 from left to right and from right to left.

PATTERN

Math / Simulation

TIME $O(n)$ **SPACE** $O(1)$

SOLUTION

C++ • 0009

```
class Solution {
public:
    bool isPalindrome(int x){
        if (x < 0)
            return false;
        if (x != 0 && x % 10 == 0)
            return false;
        int rev = 0;
        while (x > rev) {
            rev = rev * 10 + x % 10;
            x /= 10;
        }
        if(x==rev || x==rev/10)
            return true;
        else
            return false;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

6 / 31

PROBLEM 0011

11. Container With Most Water

Medium

Two Pointers

C++

QUESTION

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are `(i, 0)` and `(i, height[i])`.

EXAMPLE

Input: `height = [1,8,6,2,5,4,8,3,7]` Output: 49 Explanation: The above vertical lines are represented by array `[1,8,6,2,5,4,8,3,7]`. In this case, the max area of water (blue section) the container can contain is 49.

PATTERN

Two Pointers

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0011

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int left = 0;
        int right = height.size()-1;
        int area = 0;
        while(left<right){
            int h = min(height[left],height[right]);
            int a = h*(right-left);
            area = max(a,area);
            if(height[left]<height[right]){
                left++;
            }
            else{
                right--;
            }
        }
        return area;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

7 / 31

14. Longest Common Prefix

Easy

Arrays / Implementation

C++

QUESTION

Write a function to find the longest common prefix string amongst an array of strings.

EXAMPLE

Input: strs = ["flower","flow","flight"] Output: "fl"

PATTERN

Arrays / Implementation

TIME

O(n)

SPACE

O(1)

SOLUTION

C++ • 0014

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {
        string prefix = strs[0];
        for(int i = 1; i < strs.size(); i++) {
            int j = 0;
            while(j < prefix.size() &&
                  j < strs[i].size() &&
                  prefix[j] == strs[i][j]) {
                j++;
            }
            prefix = prefix.substr(0, j);
            if(prefix == "")
                return "";
        }
        return prefix;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

8 / 31

19. Remove Nth Node From End of List

Medium

Linked List

C++

QUESTION

Given the head of a linked list, remove the nth node from the end of the list and return its head.

EXAMPLE

Input: head = [1,2,3,4,5], n = 2 Output: [1,2,3,5]

PATTERN
Linked List

TIME
O(n)

SPACE
O(1)

SOLUTION

C++ • 0019

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode *dummy = new ListNode(0);
        dummy->next = head;
        ListNode *temp=head;
        int count = 0;
        while(temp!=NULL){
            temp = temp->next;
            count++;
        }
        int k=count-n;
        temp = dummy;
        while(k>0){
            temp=temp->next;
            k--;
        }
        ListNode *temp1=temp->next;
        temp->next=temp1->next;
        delete temp1;
        ListNode* newHead = dummy->next;
        delete dummy;
        return newHead;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

PROBLEM 0021

21. Merge Two Sorted Lists

Easy

Linked List

C++

QUESTION

You are given the heads of two sorted linked lists list1 and list2.

EXAMPLE

Input: list1 = [1,2,4], list2 = [1,3,4] Output: [1,1,2,3,4,4]

PATTERN
Linked List

TIME
O(n)

SPACE
O(1)

SOLUTION

C++ • 0021

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if(list1==NULL){return list2;}
        if(list2==NULL){return list1;}
        ListNode* dummy=new ListNode(-1);
        ListNode* temp = dummy;
        while(list1!=NULL && list2!=NULL){
            if(list1->val<=list2->val){
                temp->next=list1;
                list1=list1->next;
            }
            else{
                temp->next=list2;
                list2=list2->next;
            }
            temp=temp->next;
        }
        if(list1!=NULL){
            temp->next=list1;
        }
        else{
            temp->next=list2;
        }
        return dummy->next;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

10 / 31

24. Swap Nodes in Pairs

Medium

Linked List

C++

QUESTION

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

PATTERN
Linked List

TIME
 $O(n)$

SPACE
 $O(1)$

SOLUTION

C++ • 0024

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode* dummy = new ListNode(0);
        dummy->next = head;
        ListNode* temp = dummy;
        while(temp->next != NULL && temp->next->next != NULL) {
            ListNode* first = temp->next;
            ListNode* second = first->next;
            first->next = second->next;
            second->next = first;
            temp->next = second;
            temp = first;
        }
        ListNode* newHead = dummy->next;
        delete dummy;
        return newHead;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

11 / 31

PROBLEM 0033

33. Search in Rotated Sorted Array

Medium

Arrays / Implementation

C++

QUESTION

There is an integer array `nums` sorted in ascending order (with distinct values).

EXAMPLE

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0` Output: 4

PATTERN

Arrays / Implementation

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0033

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        for(int i=0;i<nums.size();i++){
            if(nums[i]==target){
                return i;
            }
        }
        return -1;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

12 / 31

PROBLEM 0034

34. Find First and Last Position of Element in Sorted Array

Medium

Binary Search

C++

QUESTION

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

EXAMPLE

Input: `nums = [5,7,7,8,8,10]`, `target = 8` Output: `[3,4]`

PATTERN

Binary Search

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0034

```
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int first = -1;
        int last = -1;
        // Find first occurrence
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                first = mid;
                right = mid - 1; // search left
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        // Find last occurrence
        left = 0;
        right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                last = mid;
                left = mid + 1; // search right
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        return {first, last};
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

13 / 31

PROBLEM 0035

35. Search Insert Position

Easy

Binary Search

C++

QUESTION

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

EXAMPLE

Input: nums = [1,3,5,6], target = 5 Output: 2

PATTERN
Binary Search

TIME
O(n)

SPACE
O(1)

SOLUTION

C++ • 0035

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size()-1;
        while(left<=right){
            int mid = left + (right - left)/2;
            if(nums[mid]==target){
                return mid;
            }
            if(target>nums[mid]){
                left=mid+1;
            }
            else{
                right=mid-1;
            }
        }
        return left;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

14 / 31

49. Group Anagrams

Medium

Hashing / Set

C++

QUESTION

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

PATTERN

Hashing / Set

TIME $O(n \log n)$ **SPACE** $O(n) / O(1)$

SOLUTION

C++ • 0049

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> mp;
        for(string s: strs){
            string key = s;
            sort(key.begin(), key.end());
            mp[key].push_back(s);
        }
        vector<vector<string>> ans;
        for(auto x: mp){
            ans.push_back(x.second);
        }
        return ans;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

15 / 31

PROBLEM 0053

53. Maximum Subarray

Medium

Greedy / Kadane

C++

QUESTION

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

EXAMPLE

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]` Output: 6 Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

PATTERN

Greedy / Kadane

TIME $O(n)$ **SPACE** $O(1)$

SOLUTION

C++ • 0053

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int sum = 0;
        int maxsum = INT_MIN;
        for(int i = 0; i < nums.size(); i++){
            sum += nums[i];
            maxsum = max(maxsum, sum);
            if(sum < 0){
                sum = 0;
            }
        }
        return maxsum;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

16 / 31

PROBLEM 0076

76. Minimum Window Substring

Hard

Hashing / Set

C++

QUESTION

Given two strings *s* and *t* of lengths *m* and *n* respectively, return the minimum window substring of *s* such that every character in *t* (including duplicates) is included in the window. If there is no such substring, return the empty string "".

EXAMPLE

Input: *s* = "ADOBECODEBANC", *t* = "ABC" Output: "BANC" Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string *t*.

PATTERN

Hashing / Set

TIME

O(n) average

SPACE

O(n)

SOLUTION

C++ • 0076

```
class Solution {
public:
    string minWindow(string s, string t) {
        unordered_map<char,int>n,w;
        for(char c:t) n[c]++;
        int l = 0,start=0;
        int req = n.size();
        int have = 0;
        int len = INT_MAX ;
        for(int r = 0; r<s.size(); r++){
            w[s[r]]++;
            if(n.count(s[r]) && w[s[r]]==n[s[r]]){
                have++;
            }
            while(have == req){
                if(r-l+1 < len){
                    len = r-l+1;
                    start = l;
                }
                w[s[l]]--;
                if(n.count(s[l])&&w[s[l]]<n[s[l]]){
                    have--;
                }
                l++;
            }
        }
        if(len == INT_MAX) return "";
        return s.substr(start,len);
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

17 / 31

PROBLEM 0081

81. Search in Rotated Sorted Array II

Medium

Binary Search

C++

QUESTION

There is an integer array `nums` sorted in non-decreasing order (not necessarily with distinct values).

EXAMPLE

Input: `nums = [2,5,6,0,0,1,2]`, `target = 0` Output: `true`

PATTERN

Binary Search

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0081

```
class Solution {
public:
    bool search(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target)
                return true;
            // Duplicates: cannot determine which half is sorted
            if (nums[left] == nums[mid] && nums[mid] == nums[right]) {
                left++;
                right--;
            }
            // Left half is sorted
            else if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            }
            // Right half is sorted
            else {
                if (nums[mid] < target && target <= nums[right])
                    left = mid + 1;
                else
                    right = mid - 1;
            }
        }
        return false;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

18 / 31

94. Binary Tree Inorder Traversal

Easy

Binary Search

C++

QUESTION

Given the root of a binary tree, return the inorder traversal of its nodes' values.

PATTERN

Binary Search

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0094

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<int> ans;
    vector<int> inorderTraversal(TreeNode* root) {
        if(root == NULL)
            return ans;
        inorderTraversal(root->left);
        ans.push_back(root->val);
        inorderTraversal(root->right);
        return ans;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

PROBLEM 0098

98. Validate Binary Search Tree

Medium

Binary Search

C++

QUESTION

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

EXAMPLE

Input: root = [2,1,3] Output: true

PATTERN

Binary Search

TIME

O(n)

SPACE

O(1)

SOLUTION

C++ • 0098

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool check(TreeNode* root, long long low, long long high) {
        if (root == NULL) return true;
        if (root->val <= low || root->val >= high)
            return false;
        return check(root->left, low, root->val) && check(root->right, root->val, high);
    }
    bool isValidBST(TreeNode* root) {
        return check(root, LLONG_MIN, LLONG_MAX);
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

20 / 31

PROBLEM 0100

100. Same Tree

Easy

Binary Search

C++

QUESTION

Given the roots of two binary trees *p* and *q*, write a function to check if they are the same or not.

EXAMPLE

Input: *p* = [1,2,3], *q* = [1,2,3] Output: true

PATTERN

Binary Search

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0100

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if(p==NULL && q==NULL) return true;
        if(p==NULL || q==NULL) return false;
        if(p->val!=q->val) return false;
        return isSameTree(p->left,q->left)&&isSameTree(p->right,q->right);
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

21 / 31

PROBLEM 0101

101. Symmetric Tree

Easy

Binary Search

C++

QUESTION

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

EXAMPLE

Input: root = [1,2,2,3,4,4,3] Output: true

PATTERN
Binary Search

TIME
O(n)

SPACE
O(1)

SOLUTION

C++ • 0101

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool mirror(TreeNode* left, TreeNode* right) {
        if (left == NULL && right == NULL)
            return true;
        if (left == NULL || right == NULL)
            return false;
        if (left->val != right->val)
            return false;
        return mirror(left->left, right->right) &&
            mirror(left->right, right->left);
    }
    bool isSymmetric(TreeNode* root) {
        if (root == NULL)
            return true;
        return mirror(root->left, root->right);
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

22 / 31

PROBLEM 0121

121. Best Time to Buy and Sell Stock

Easy

Greedy / Kadane

C++

QUESTION

You are given an array prices where prices[i] is the price of a given stock on the ith day.

EXAMPLE

Input: prices = [7,1,5,3,6,4] Output: 5 Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

PATTERN

Greedy / Kadane

TIME

O(n)

SPACE

O(1)

SOLUTION

C++ • 0121

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minprice = prices[0];
        int profit = 0;
        for(int i =1;i<prices.size();i++){
            minprice= min(minprice,prices[i]);
            profit = max(profit,prices[i]-minprice);
        }
        return profit;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

23 / 31

PROBLEM 0122

122. Best Time to Buy and Sell Stock II

Medium

Greedy / Kadane

C++

QUESTION

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.

EXAMPLE

Input: prices = [7,1,5,3,6,4] Output: 7 Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4. Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3. Total profit is 4 + 3 = 7.

PATTERN

Greedy / Kadane

TIME

O(n)

SPACE

O(1)

SOLUTION

C++ • 0122

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int profit=0;
        for(int i = 1;i<prices.size();i++){
            if(prices[i]>prices[i-1]){
                profit = profit + prices[i] - prices[i-1];
            }
        }
        return profit;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

24 / 31

PROBLEM 0128

128. Longest Consecutive Sequence

Medium

Hashing / Set

C++

QUESTION

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

EXAMPLE

Input: `nums = [100,4,200,1,3,2]` Output: 4 Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

PATTERN

Hashing / Set

TIME

 $O(n)$ average

SPACE

 $O(n)$

SOLUTION

C++ • 0128

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> st(nums.begin(),nums.end());
        int ans =0;
        for(int x:st){
            if(st.find(x-1)==st.end()){
                int current =x;
                int count =1;
                while(st.find(current+1)!=st.end()){
                    current++;
                    count++;
                }
                ans=max(ans,count);
            }
        }
        return ans;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

25 / 31

PROBLEM 0189

189. Rotate Array

Medium

Math / Simulation

C++

QUESTION

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

EXAMPLE

Input: `nums = [1,2,3,4,5,6,7]`, `k = 3` Output: `[5,6,7,1,2,3,4]` Explanation: rotate 1 steps to the right: `[7,1,2,3,4,5,6]` rotate 2 steps to the right: `[6,7,1,2,3,4,5]` rotate 3 steps to the right: `[5,6,7,1,2,3,4]`

PATTERN

Math / Simulation

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0189

```
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        k = k % nums.size();
        reverse(nums.begin(), nums.end());
        reverse(nums.begin(), nums.begin() + k);
        reverse(nums.begin() + k, nums.end());
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

26 / 31

PROBLEM 0209

209. Minimum Size Subarray Sum

Medium

Arrays / Implementation

C++

QUESTION

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return 0 instead.

EXAMPLE

Input: `target = 7, nums = [2,3,1,2,4,3]` Output: 2 Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

PATTERN

Arrays / Implementation

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0209

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int l=0;
        int sum=0;
        int len = INT_MAX;
        for(int r=0;r<nums.size();++r){
            sum += nums[r];
            while(sum>=target){
                len = min(r-l+1,len);
                sum-=nums[l];
                ++l;
            }
        }
        if(len == INT_MAX) return 0;
        return len;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

27 / 31

PROBLEM 0215

215. Kth Largest Element in an Array

Medium

Arrays / Implementation

C++

QUESTION

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

EXAMPLE

Input: `nums = [3,2,1,5,6,4]`, `k = 2` Output: 5

PATTERN

Arrays / Implementation

TIME

 $O(n)$

SPACE

 $O(1)$

SOLUTION

C++ • 0215

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;
        for(int x:nums){
            pq.push(x);
            if(pq.size()>k){
                pq.pop();
            }
        }
        return pq.top();
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

28 / 31

PROBLEM 0239

239. Sliding Window Maximum

Hard

Two Pointers

C++

QUESTION

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

EXAMPLE

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3` Output: `[3,3,5,5,6,7]` Explanation: Window position Max ----
 ----- [1 3 -1] -3 5 3 6 7 3 1 [3 -1 -3] 5 3 6 7 3 1 3 [-1 -3 5] 3 6 7 5 1 3 -1 [-3 5 3]
 6 7 5 1 3 -1 -3 [5 3 6] 7 6 1 3 -1 -3 5 [3 6 7] 7

PATTERN

Two Pointers

TIME

O(n)

SPACE

O(1)

SOLUTION

C++ • 0239

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        deque<int> dq;
        vector<int> ans;
        for (int r = 0; r < nums.size(); r++) {
            // Remove elements outside the window
            while (!dq.empty() && dq.front() <= r - k) {
                dq.pop_front();
            }
            // Remove smaller elements
            while (!dq.empty() && nums[dq.back()] <= nums[r]) {
                dq.pop_back();
            }
            // Add current index
            dq.push_back(r);
            // Window size is k
            if (r >= k - 1) {
                ans.push_back(nums[dq.front()]);
            }
        }
        return ans;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

29 / 31

PROBLEM 0455

455. Assign Cookies

Easy

Sorting

C++

QUESTION

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

EXAMPLE

Input: $g = [1,2,3]$, $s = [1,1]$ Output: 1 Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3. And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content. You need to output 1.

PATTERN
Sorting**TIME**
 $O(n \log n)$ **SPACE**
 $O(n) / O(1)$

SOLUTION

C++ • 0455

```
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());
        int i = 0;
        int j = 0;
        int count = 0;
        while (i < g.size() && j < s.size()) {
            if (s[j] >= g[i]) {
                count++;
                i++;
                j++;
            }
            else {
                j++;
            }
        }
        return count;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

30 / 31

PROBLEM 3718

4080. Smallest Missing Multiple of K

Easy

Hashing / Set

C++

QUESTION

Given an integer array `nums` and an integer `k`, return the smallest positive multiple of `k` that is missing from `nums`.

PATTERN

Hashing / Set

TIME

 $O(n)$ average

SPACE

 $O(n)$

SOLUTION

C++ • 3718

```
class Solution {
public:
    int missingMultiple(vector<int>& nums, int k) {
        unordered_set<int> st;
        for(int x : nums){
            st.insert(x);
        }
        for(int i=1;i<=nums.size()+1;i++){
            int x = k*i;
            if(st.find(x)==st.end()){
                return x;
            }
        }
        return -1;
    }
};
```

Revision cue: Before moving on, explain the core idea in one or two sentences without looking back.

31 / 31